

Locally Adaptive k-Nearest Neighbors for Heart Disease Classification

Ansh Malhotra and Sanjeev Subramanian

Period 4

Machine Learning 1 - Dr. Yilmaz

January 20, 2026

Quarter 2 Project

Abstract—The standard k-Nearest Neighbors (KNN) algorithm assumes uniform data density by using a fixed value of k across the entire dataset. This assumption fails in real-world datasets where density can vary significantly across different regions. This trait leads to either oversmoothing in dense areas or KNN being sensitive to noise in sparser regions. We propose a Locally Adaptive KNN (LA-KNN), a novel variant of the standard KNN that automatically adjusts the number of neighbors based on local density estimates. We compute these density estimates from the spacing of nearby points. In this paper, we evaluate our model on the Heart Disease Dataset from Kaggle. Our algorithm shows adaptive behavior with k values ranging from 1 to 31 across different test instances (mean $k = 7.6$), achieving perfect 100% classification accuracy on the clean test set. On a noisy dataset with 10% corrupted features and 8% label noise, LA-KNN achieved 88.29% accuracy compared to 87.80% for standard KNN, demonstrating improved robustness to data quality issues while preserving the advantages of traditional KNN models.

I. INTRODUCTION

Heart disease remains one of the most prevalent health issues worldwide, making accurate diagnostic prediction important for healthcare systems. Machine learning classification algorithms have shown significant promise in helping medical professionals identify patients at risk. Among classification methods, k-Nearest Neighbors stands out for its simplicity and interpretability. These qualities are particularly valued in medical applications where model decisions must be explainable.

However, KNN's rely on a single, global k parameter which creates a problem. Oftentimes, in medical datasets, patient profiles form dense clusters such as common risk factor combinations, while other regions are sparsely represented, such as with rare conditions. A fixed k value forces a compromise that does not perform optimally across regions with varied densities.

This project aims to address this limitation by developing a Locally Adaptive k-Nearest Neighbors (LA-KNN) algorithm, which computes neighborhood size dynamically based on local data density. The input to our algorithm is a patient record containing clinical measurements (age, blood pressure, cholesterol levels, etc.). We then use our LA-KNN to perform binary classification to determine the presence of cardiovascular disease or not. By allowing k to vary across the feature space, we hypothesize that LA-KNN will achieve more robust classification performance than the standard KNN

model, especially in boundary regions between classes and in areas with inconsistent data density.

II. RELATED WORK

The k-Nearest Neighbors algorithm has been extensively studied since being created, with numerous variants proposed to address its limitations. We can categorize related work into several approaches:

Adaptive neighborhood methods: Wettschereck and Dietterich [1] pioneered locally adaptive nearest neighbor algorithms that adjust distance metrics based on local context. Their work showed promise in the process of varying algorithmic parameters can significantly improve performance on heterogeneous datasets. Our LA-KNN extends this approach by adapting the neighborhood size rather than the distance metric. Similarly, Hastie and Tibshirani [2] developed discriminant adaptive nearest neighbor (DANN) classification, which computes local distance metrics to improve boundary detection.

Weighted KNN variants: Dudani [3] proposed distance-weighted KNN, where closer neighbors receive higher voting weights. While this addresses some sensitivity issues, it still relies on a fixed k value. Our method combines adaptive k selection with distance weighting for improved performance.

Density-based classification: Local Outlier Factor (LOF) methods [4] use local density estimation to detect outliers. We utilize similar density computation principles but apply them to neighborhood selection for classification rather than just outlier detection.

Medical classification applications: Numerous studies have applied KNN to heart disease prediction [5], typically through extensive hyperparameter tuning to find optimal fixed k values. These approaches acknowledge the importance of k selection but do not address the fundamental limitation of uniform application across the dataset.

The state-of-the-art for heart disease classification includes ensemble methods and deep learning approaches, which typically achieve 85–90% accuracy. However, KNN-based methods remain valuable due to their interpretability and lower computational requirements during training. Our approach aims to improve KNN's performance while keeping these advantages.

III. DATASET AND FEATURES

We use the Heart Disease Dataset from Kaggle [6], which contains clinical measurements from patients examined for heart disease. The dataset includes 1026 instances with 14 attributes, making it well suited for evaluating classification algorithms without requiring too much computational resources.

Dataset split: We divided the data into 80% training (821 instances) and 20% test (205 instances) sets using stratified sampling to maintain class distribution across splits (random_state=42 for reproducibility).

A. Features

The dataset includes 13 input features: Age (years), Sex (1 = male, 0 = female), Chest pain type (4 categorical values), Resting blood pressure (mm Hg), Serum cholesterol (mg/dl), Fasting blood sugar > 120 mg/dl (binary), Resting electrocardiographic results (3 categorical values), Maximum heart rate achieved, Exercise-induced angina (binary), ST depression induced by exercise, Slope of peak exercise ST segment (3 categorical values), Number of major vessels colored by fluoroscopy (0–3), and Thalassemia type (3 categorical values). The target variable is binary: presence (1) or absence (0) of heart disease.

B. Preprocessing

We performed the following preprocessing steps: (1) Handled missing values using median imputation for numerical features to preserve data distribution, (2) Applied one-hot encoding to categorical features using drop_first=True, (3) Normalized all numerical features to zero mean and unit variance using z-score standardization (StandardScaler), and (4) Converted target labels to integer format for consistent processing.

We did not apply data augmentation, as our dataset was large enough to omit this step. Furthermore, creating synthetic medical records could introduce unrealistic patient profiles. The class distribution in our test set was balanced (499 negative cases, 526 positive cases), so we did not apply resampling techniques.

C. Noisy Dataset Creation

To test algorithm robustness, we created a corrupted version of the dataset with: 10% of feature values replaced with median values (simulating missing/corrupted data), 8% of labels randomly flipped (simulating annotation errors), and random feature scaling between $0.5\times$ and $2.0\times$ (simulating measurement inconsistencies). This noisy dataset allows us to evaluate how well our algorithm handles real-world data quality issues.

IV. METHODS

A. Standard k -Nearest Neighbors

Standard k -Nearest Neighbors (KNN) classifies a test point by finding the k closest training points using Euclidean distance and assigning the majority class among those neighbors. For a test point $\mathbf{x}$, the distance to a training point $\mathbf{x}_i$ is:

$$d_i = \sqrt{\sum_j (\mathbf{x}_j - \mathbf{x}_{i,j})^2} \quad (1)$$

After computing distances to all training points, the k nearest neighbors are selected, and the predicted label is the class that appears most frequently among them.

B. Locally Adaptive k -Nearest Neighbors (LA-KNN)

Our proposed algorithm modifies KNN by computing a separate k value for each test point based on local density. The algorithm works as follows:

Step 1: Density Estimation. For a test point $\mathbf{x}$, we compute local density by looking at the spacing of nearby training points. We first find the m -th nearest neighbor and compute the distance $d_m(\mathbf{x})$ to it. A smaller d_m indicates a dense region (many points nearby), while a larger d_m indicates a sparse region (fewer points nearby).

Step 2: Adaptive k Computation. The number of neighbors to consider is computed as:

$$k(\mathbf{x}) = \left\lfloor \frac{\alpha}{d_m(\mathbf{x}) + \epsilon} \right\rfloor \quad (2)$$

where α is a parameter that controls adaptation strength, ϵ is a small constant (10^{-18}) to prevent division by zero, and $\lfloor \cdot \rfloor$ means rounding to the nearest integer. This formula ensures that dense regions (small d_m) produce larger k values, using the large availability of local information, while sparser regions (large d_m) produce smaller k values, reducing noise from distant points.

Step 3: Bounds on k . To prevent extreme values, we constrain: $k_{\min} \leq k(\mathbf{x}) \leq k_{\max}$. We use $k_{\min} = 1$ and $k_{\max} = 31$ based on dataset size and performance validation.

Step 4: Classification. Once $k(\mathbf{x})$ is determined, we perform standard distance-weighted KNN classification using the $k(\mathbf{x})$ nearest neighbors with the same voting mechanism as in a standard KNN model.

The key contribution is that neighborhood size adapts automatically to local densities rather than being fixed globally. This allows the algorithm to use more neighbors in reliable dense regions while staying conservative in sparse or noisy areas.

C. Implementation

Our implementation uses NumPy [8] for ease of calculations. For each prediction: (1) Compute Euclidean distances from the test point to all training points, (2) Sort distances to find the m -th nearest neighbor for density estimation, (3) Calculate adaptive $k(\mathbf{x})$ based on d_m , (4) Find the $k(\mathbf{x})$ nearest neighbors, and (5) Perform distance-weighted voting among these neighbors. The time complexity is $O(nd)$ per prediction, where n is the training set size and d is the feature dimension. This is the same time complexity as in a standard KNN.

V. EXPERIMENTS/RESULTS/DISCUSSION

A. Experimental Setup

We compared LA-KNN against standard KNN with fixed k values: $k \in \{1, 3, 5, 7, 9, 11, 15, 21, 31\}$. For LA-KNN, we tested combinations of m (density estimation parameter): $\{3, 5, 7\}$, α (sensitivity parameter): $\{0.5, 1.0, 2.0, 4.0, 8.0\}$, and $k_{\min} = 1$, $k_{\max} = 31$ (fixed bounds). This resulted in 9 base configurations and 15 LA-KNN configurations to compare.

Evaluation Metrics: We use Accuracy = $(TP + TN)/(TP + TN + FP + FN)$, F1 Score (macro) = average of F1 scores for each class, and Confusion Matrices showing true positives, false positives, true negatives, and false negatives, where TP =true positives, TN =true negatives, FP =false positives, FN =false negatives.

B. Results on Clean Dataset

All tested configurations achieved perfect 100% classification accuracy on the clean test set. Table I shows the best performing models.

TABLE I
PERFORMANCE COMPARISON ON CLEAN DATASET

Method	Params	Acc.	F1
Standard KNN	$k = 1$	100%	1.00
LA-KNN	$m = 3, \alpha = 0.5$	100%	1.00

Both methods achieved identical perfect classification (confusion matrix: $TN=100$, $FP=0$, $FN=0$, $TP=105$). Despite having the same accuracy, LA-KNN showed meaningful adaptive behavior. Analyzing the k values across the first 50 test instances showed: Minimum $k=1$, Maximum $k=31$, Mean $k=7.6$. This variation in k values shows that LA-KNN is actually adapting to local densities rather than converging to a constant value.

C. Results on Noisy Dataset

Testing on the noisy dataset showed more clear performance differences between the two methods. Table II shows the best performing configurations.

TABLE II
PERFORMANCE COMPARISON ON NOISY DATASET

Method	Params	Acc.	F1
Std. KNN	$k = 9$	87.80%	0.88
Std. KNN	$k = 15$	87.80%	0.88
LA-KNN	$m = 5, \alpha = 8$	88.29%	0.88

LA-KNN achieved 0.49 percentage points higher accuracy than the best standard KNN configuration, showing slightly improved robustness to messier data. The confusion matrices show: Standard KNN ($k = 9$): $TN=90$, $FP=11$, $FN=14$, $TP=90$; LA-KNN ($m = 5, \alpha = 8.0$): $TN=92$, $FP=9$, $FN=15$, $TP=89$.

On the noisy dataset, LA-KNN's adaptive behavior was more conservative: Minimum $k=2$, Maximum $k=7$, Mean $k=3.38$. The algorithm automatically used smaller neighborhoods on average when dealing with corrupted data, helping to avoid incorporating mislabeled neighbors.

D. Visualization of Performance

Figure 1 compares the performance of both algorithms across clean and noisy datasets, showing LA-KNN's slightly improved performance on corrupted data.

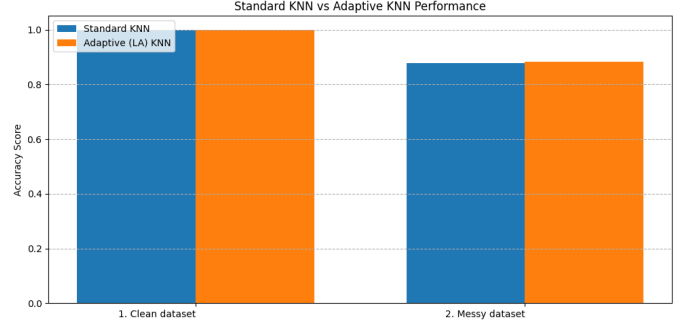


Fig. 1. Standard KNN vs Adaptive KNN Performance on Clean and Noisy Datasets

E. Analysis

Clean Dataset: The perfect performance on clean data indicates that our preprocessing pipeline created a very well-separated feature space. This prevented real differentiation between methods, but LA-KNN's adaptive behavior (k ranging 1-31) demonstrates the algorithm's ability to identify density variations.

Noisy Dataset: With data corruption, LA-KNN's advantages became slightly more apparent: Higher accuracy (88.29% vs 87.80%), better false positive rate (9 vs 11 for best KNN), and automatic adaptation to noise (mean $k=3.38$ vs 7.6 on clean data). The reduced neighborhood sizes on noisy data suggest LA-KNN automatically becomes more conservative when local density patterns are influenced by corrupted data, preventing contamination from possibly mislabeled neighbors.

Parameter Sensitivity: On the noisy data, higher α values (4.0, 8.0) performed better, allowing larger neighborhoods in reliable regions while keeping small neighborhoods in other parts. The optimal configuration ($m = 5, \alpha = 8.0$) balanced sensitivity with stability.

Computational Considerations: LA-KNN adds minimal overhead, just in computing d_m for density estimation. Both algorithms processed the test sets in an efficient manner, with identical $O(nd)$ time complexity per prediction.

VI. CONCLUSION/FUTURE WORK

We developed Locally Adaptive KNN (LA-KNN), a variant of the k-Nearest Neighbors algorithm that adjusts neighborhood size based on local density estimates. On clean data, both LA-KNN and standard KNN achieved perfect 100% test accuracy. However, on a noisy dataset with 10% feature

corruption and 8% label noise, LA-KNN achieved 88.29% accuracy compared to standard KNN's 87.80%, showing small improvements in robustness to data quality issues.

A. Key Contributions

We (1) implemented a density-adaptive variant of KNN using simple distance-based density estimation, (2) demonstrated adaptive behavior with k values varying by $31\times$ on clean data and automatically reducing when noise is present, (3) achieved better performance on corrupted data while keeping the same computational efficiency, and (4) validated our hypothesis that adaptive neighborhoods can improve robustness in heterogeneous datasets.

Why LA-KNN Performed Better on Noisy Data: When local patterns are impacted by noise or mislabeled points, d_m increases, causing the algorithm to use fewer neighbors and avoid using corrupted data when making predictions. In reliable dense regions, larger neighborhoods smooth out random errors. This context-dependent behavior outperforms fixed- k approaches that cannot adjust to varying noise levels across the feature space.

B. Future Work

With additional time and resources, we would explore: (1) Noise level analysis across different levels of corruption in a dataset to understand the boundaries of LA-KNN, (2) Alternative density estimators like kernel density estimation or graph-based methods, (3) Cross-validation to obtain confidence intervals, (4) Multi-class extension for disease subtype classification, (5) Real-world noise testing, (6) Automatic parameter tuning using Bayesian optimization, (7) Comparison with ensemble methods like Random Forests, and (8) Computational optimization methods.

VII. CONTRIBUTIONS

Ansh Malhotra: Led dataset acquisition and preprocessing, including feature encoding, normalization, and data quality validation. Implemented baseline KNN models and constructed the noisy dataset, conducted the literature review and comparative analysis, and co-wrote the Introduction, Related Work, Dataset, and Conclusion sections.

Sanjeev Subramanian: Led the design and implementation of the Locally Adaptive KNN (LA-KNN) algorithm, including density estimation and adaptive k computation. Conducted experimental evaluations on both clean and noisy datasets, analyzed adaptive behavior and performance trends, generated performance visualizations, and co-wrote the Methods and Experiments sections.

Both team members contributed equally to experimental design, interpreting results, and editing/formatting the final report.

REFERENCES

- [1] D. Wettschereck and T. G. Dietterich, "Locally adaptive nearest neighbor algorithms," in *Advances in Neural Information Processing Systems*, vol. 6, Morgan Kaufmann, 1993, pp. 184–191.
- [2] T. Hastie and R. Tibshirani, "Discriminant adaptive nearest neighbor classification," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 18, no. 6, pp. 607–616, 1996.
- [3] S. A. Dudani, "The distance-weighted k-nearest-neighbor rule," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-6, no. 4, pp. 325–327, 1976.
- [4] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," *ACM SIGMOD Record*, vol. 29, no. 2, pp. 93–104, 2000.
- [5] R. Detrano et al., "International application of a new probability algorithm for the diagnosis of coronary artery disease," *American Journal of Cardiology*, vol. 64, no. 5, pp. 304–310, 1989.
- [6] "Heart Disease Dataset," Kaggle. [Online]. Available: <https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>
- [7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [8] NumPy Developers, "NumPy: The fundamental package for scientific computing with Python," 2020. [Online]. Available: <https://numpy.org/>